Stage 3.2 — Pydantic Settings & Configuration Validation

Enterprise RAG Architecture Configuration Management Guide

Absolutely. Now we move from "YAML configuration works" to "YAML configuration is validated and type-safe."
This is a very important step for your Enterprise RAG architecture.

Our current flow is:

```
YAML files
    ↓
PyYAML
    ↓
Python dictionary
    ↓
Settings classes
    ↓
RAG application
```

The problem is that our current Settings classes don't validate the data.
For example, this YAML:

```yaml
llm:
  provider: Groq
  model: llama-3.1-8b-instant
  temperature: hello
```

will load successfully.
But:

```
temperature = "hello"
```

is obviously wrong.
Pydantic will solve this.

1. What is Pydantic?

Pydantic allows us to define:

What configuration should look like and what data types it must contain.
For example:

```python
class LLMSettings(BaseModel):
    provider: str
    model: str
    temperature: float
```

This tells Pydantic:

- `provider` → must be a string

- `model` → must be a string
- `temperature` → must be a number

So we're moving from:

```
config["llm"]["model"]
```

to a strongly defined configuration model.

2. Install Pydantic

From your VS Code terminal, make sure your virtual environment is activated and run:

```
pip install pydantic
```

Then verify:

```
pip show pydantic
```

You should see the installed version.

3. Our project structure

We don't need to change the overall structure:

```
Stage 3 YAML-Based Application Configuration/
│
├── config/
│ ├── app.yaml
│ ├── llm.yaml
│ ├── embedding.yaml
│ ├── vector_database.yaml
│ ├── retrieval.yaml
│ ├── chunking.yaml
│ └── logging.yaml
│
├── src/
│ ├── __init__.py
│ ├── yaml_loader.py
│ ├── config_loader.py
│ └── settings.py ← We'll modify this
│
└── main.py
```

For this lesson, we only need to modify:

```
src/settings.py
```

and later test using:

```
main.py
```

4. Replace your current settings.py

File: `src/settings.py`

Replace your current Stage 3.1 implementation with:

```
from pathlib import Path

from pydantic import BaseModel

from src.config_loader import load_all_configs


# ============================================================
# Application Settings
# ============================================================

class ApplicationSettings(BaseModel):

    # Application name must be a string.
    name: str

    # Application version must be a string.
    version: str

    # Environment must be a string.
    environment: str


# ============================================================
# LLM Settings
# ============================================================

class LLMSettings(BaseModel):

    # LLM provider, for example Groq.
    provider: str

    # LLM model name.
    model: str

    # Temperature must be a floating-point number.
    temperature: float


# ============================================================
# Embedding Settings
# ============================================================

class EmbeddingSettings(BaseModel):

    # Embedding provider.
    provider: str

    # Embedding model.
    model: str


# ============================================================
# Vector Database Settings
# ============================================================

class VectorDatabaseSettings(BaseModel):

    # Vector database provider.
    provider: str

    # Database directory.
```

```
    directory: str

    # Collection name.
    collection_name: str


# ============================================================
# Retrieval Settings
# ============================================================

class RetrievalSettings(BaseModel):

    # Number of documents to retrieve.
    top_k: int

    # Whether reranking is enabled.
    reranking: bool


# ============================================================
# Chunking Settings
# ============================================================

class ChunkingSettings(BaseModel):

    # Chunking strategy.
    strategy: str

    # Size of each chunk.
    chunk_size: int

    # Number of overlapping characters/tokens.
    chunk_overlap: int


# ============================================================
# Logging Settings
# ============================================================

class LoggingSettings(BaseModel):

    # Logging level.
    level: str

    # Logging file path.
    file: str


# ============================================================
# Main Settings
# ============================================================

class Settings(BaseModel):

    # Application configuration.
    application: ApplicationSettings

    # LLM configuration.
    llm: LLMSettings

    # Embedding configuration.
    embedding: EmbeddingSettings
```

```
    # Vector database configuration.
    vector_database: VectorDatabaseSettings

    # Retrieval configuration.
    retrieval: RetrievalSettings

    # Chunking configuration.
    chunking: ChunkingSettings

    # Logging configuration.
    logging: LoggingSettings


# ============================================================
# Locate project root
# ============================================================

# __file__ = src/settings.py
#
# parent      = src/
# parent.parent = project root
BASE_DIR = Path(__file__).resolve().parent.parent


# ============================================================
# Locate configuration directory
# ============================================================

CONFIG_DIR = BASE_DIR / "config"


# ============================================================
# Load all YAML configuration files
# ============================================================

# This returns one combined Python dictionary.
_CONFIG = load_all_configs(CONFIG_DIR)


# ============================================================
# Create validated Settings object
# ============================================================

# Pydantic validates the entire configuration here.
settings = Settings(**_CONFIG)
```

5. Understand the most important change

Previously we had:

```
class LLMSettings:

    def __init__(self, config):
        self.provider = config["provider"]
        self.model = config["model"]
        self.temperature = config["temperature"]
```

Now:

```
class LLMSettings(BaseModel):

    provider: str
    model: str
    temperature: float
```

This is much more powerful.
We are declaring a schema.

6. What does BaseModel do?

This:

```
class LLMSettings(BaseModel):
```

means:

`LLMSettings` is a Pydantic data model.
And:

```
temperature: float
```

means:

`temperature` is expected to be a number.
Similarly:

```
top_k: int
```

means:

`top_k` should be an integer.
And:

```
reranking: bool
```

means:

`reranking` should be a Boolean.

7. The important line

At the bottom of `settings.py`:

```
settings = Settings(**_CONFIG)
```

This is the point where Pydantic validates your complete configuration.
Suppose `_CONFIG` looks like:

```
{
    "application": {
        "name": "Enterprise RAG application",
        "version": 1.0,
        "environment": "development"
    },

    "llm": {
        "provider": "Groq",
        "model": "llama-3.1-8b-instant",
        "temperature": 0
    },

    ...
}
```

Pydantic takes that dictionary and builds:

```
Settings
│
├── ApplicationSettings
├── LLMSettings
├── EmbeddingSettings
├── VectorDatabaseSettings
├── RetrievalSettings
├── ChunkingSettings
└── LoggingSettings
```

8. Test your existing configuration

Your existing YAML should work.

Run:

```
python main.py
```

You should still get output similar to:

```
Application: Enterprise RAG application
Version: 1.0
Environment: development

LLM Provider: Groq
LLM Model: llama-3.1-8b-instant
LLM Temperature: 0

Embedding Provider: sentence-transformers
Embedding Model: all-MiniLM-L6-v2

Vector DB: chroma
ChromaDB Directory: ./chroma_db
Collection: enterprise_knowledge_base

Top K: 5
Reranking: False

Chunking Strategy: recursive
Chunk Size: 1000
Chunk Overlap: 200

Log Level: INFO
Log File: logs/application.log
```

9. Now let's deliberately create an error

This is where the learning becomes interesting.
Open:

```
config/llm.yaml
```

Temporarily change:

```
temperature: 0
```

to:

```
temperature: hello
```

So:

```
llm:
  provider: Groq
  model: llama-3.1-8b-instant
  temperature: hello
```

Now run:

```
python main.py
```

Pydantic should raise a validation error.
You'll see something similar to:

```
ValidationError

llm.temperature
Input should be a valid number
```

The exact error wording depends on your Pydantic version.

This is exactly what we wanted.
Instead of allowing bad configuration to travel deep into your RAG application, the application fails at configuration loading time.

10. Another experiment — top_k

Open:

```
config/retrieval.yaml
```

Change:

```
top_k: 5
```

to:

```
top_k: hello
```

Run:

```
python main.py
```

You should get a validation error for:

```
retrieval.top_k
```

because we declared:

```
top_k: int
```

11. Test a Boolean

Change:

```
reranking: false
```

to:

```
reranking: hello
```

Again:

```
python main.py
```

Pydantic should reject the invalid value.

12. Restore your YAML files

After experimenting, restore:

`config/llm.yaml`

```
llm:
  provider: Groq
  model: llama-3.1-8b-instant
  temperature: 0
```

`config/retrieval.yaml`

```
retrieval:
  top_k: 5
  reranking: false
```

Run:

```
python main.py
```

Everything should work again.

13. One very interesting thing Pydantic does

Consider:

```
temperature: 0
```

Our model says:

```
temperature: float
```

Depending on Pydantic's normal coercion rules, Pydantic can convert compatible numeric input into the declared type.
So you may find:

```
settings.llm.temperature
```

behaves as a numeric value even though YAML represented 0 as an integer.

You can verify:

```
print(type(settings.llm.temperature))
```

You should get a floating-point type.
Likewise:

```
print(type(settings.retrieval.top_k))
```

should show:

```
<class 'int'>
```

This is another benefit of having a typed configuration model.

14. Why this is much closer to production

Without validation:

```
YAML → Dictionary → Application → 💥 Error somewhere deep in the application
```

With Pydantic:

```
YAML → PyYAML → Dictionary → Pydantic validation
                                    |
                    ├── ❌ Invalid → fail immediately
                    └── ✅ Valid → Settings object → Enterprise RAG
```

This is called **fail-fast configuration validation**.
That's exactly what you want.

15. Your Enterprise RAG becomes much cleaner

Imagine your real RAG application.

`llm_service.py`

```
from src.settings import settings


model_name = settings.llm.model
temperature = settings.llm.temperature
```

`embedding_service.py`

```
from src.settings import settings


model_name = settings.embedding.model
```

`retriever.py`

```
from src.settings import settings


top_k = settings.retrieval.top_k
reranking = settings.retrieval.reranking
```

`chroma_service.py`

```
from src.settings import settings


collection_name = settings.vector_database.collection_name
```

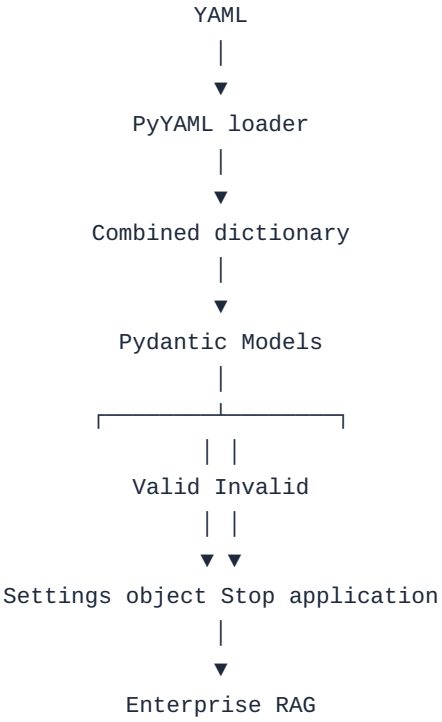
Every component gets configuration through:

```
settings
```

And settings has already been validated.

16. We've now improved the architecture considerably

Your configuration architecture is now:



And the individual models are:

```
Settings
│
├── ApplicationSettings
│
├── LLMSettings
│
├── EmbeddingSettings
│
├── VectorDatabaseSettings
│
├── RetrievalSettings
│
├── ChunkingSettings
│
└── LoggingSettings
```

17. One important correction to our terminology

At this point we're using Pydantic, but we're not yet using `pydantic-settings`.
Those are related but different things.

Pydantic
We're using it now for:
Validating and structuring the YAML configuration.

Pydantic Settings
We'll encounter it when we want:
Environment variables + .env + secrets + configuration management.

That's going to be especially relevant for your:

- `GROQ_API_KEY`
- `OPENAI_API_KEY`
- `AZURE_OPENAI_API_KEY`

because those should not go into your YAML files.

🎯 Stage 3.2 practical exercise

Don't just run the application once. Do these four experiments:

Experiment 1 — valid configuration

`temperature: 0`

Run: `python main.py`

✅ Should work.

Experiment 2 — invalid temperature

`temperature: hello`

Run: `python main.py`

❌ Pydantic should reject it.

Experiment 3 — invalid top_k

`top_k: hello`

Run: `python main.py`

❌ Pydantic should reject it.

Experiment 4 — restore everything

Restore:

`temperature: 0`

`top_k: 5`

`reranking: false`

Run: `python main.py`

✅ Should work.

If you complete those experiments, you've learned the most important concept of Stage 3.2:

YAML defines the configuration, PyYAML loads it, Pydantic validates it, and the Settings object exposes it safely to the application.

Next, we'll move to Stage 3.3 — combining YAML configuration with .env and secrets, where we'll finally connect this architecture to the GROQ_API_KEY from your real Enterprise RAG settings.py.